

SIE

Skill-to-Income AI Engine

Master Execution Plan & Team Handbook

"From Learning to Earning — Engineered, Not Left to Chance."

0. How to Use This Document

This handbook turns the SIE project documentation into a single, sequential execution plan for the team. It answers three questions for every member, at every stage: what to build, who owns it, and what must be confirmed with the rest of the team before moving on.

Read it top to bottom once as a team before writing any code. After that, use it as a living reference:

- Sections 1–2 give the project snapshot and team ownership (roles are assigned as Member 1 / 2 / 3 — final name-to-role mapping is pending).
- Section 3 is the big-picture roadmap — the order things get built in.
- Section 4 breaks that roadmap into 23 detailed phases with owners and outputs.
- Section 5 is the Responsibility Matrix — a one-page view of who owns what.
- Section 6 is the detailed, tickable checklist for each member, phase by phase, with dependency checks.
- Sections 7–9 cover integration, daily standups, and final submission readiness.
- Section 10 shows the team collaboration timeline and mandatory sync gates.
- Section 11 is the full 30-step phase-by-phase table (the master reference).
- Section 12 lists the golden rules — what to do the moment something shared (API, schema, field) changes.
- Section 13 gives a scope recommendation (MVP vs. advanced) to keep the project achievable.

1. Project Snapshot (Context)

This section summarises the SIE Project Documentation (Sections 3–11 of the reference PDF) so the whole team has the same mental model of what is being built, before diving into execution.

1.1 What SIE Is

The Skill-to-Income AI Engine (SIE) is a fully automated income execution pipeline. It takes a skill as input and produces a live, deployed income stream as output — with zero dependence on paid tools, paid APIs, or human intermediaries. It is not a recommendation engine and not a job board.

1.2 The Problem It Solves

India produces ~1.5 million engineering graduates a year, but only 20–25% are directly employable in core technical roles. The rest are not unskilled — they are unmapped: they have skills but no infrastructure to convert those skills into income. Every existing EdTech platform (Coursera, Udemy, LinkedIn Learning) stops at knowledge delivery; every freelance marketplace (Fiverr, Upwork) assumes the user already knows how to sell. Nothing bridges the two. SIE is that bridge.

1.3 The Five Core Modules

- Skill Decomposition Engine — breaks a broad skill (e.g. 'Python') into 50+ specific, sellable micro-services.
- Market Intelligence Layer — scrapes Fiverr, Upwork, LinkedIn Jobs and GitHub Trending (no paid APIs) and scores micro-skills by demand vs. competition.
- Micro-Opportunity Generator — cross-references the user's skill profile with live market data to produce a ranked list of income opportunities.
- Execution Blueprint (Income Kit) — auto-generates a gig listing, a GitHub portfolio project, a micro landing page, and cold outreach scripts.
- Adaptive Feedback Loop — learns from user-reported outcomes, click tracking, and A/B testing to keep refining pricing, positioning and strategy.

1.4 Measurable Goal

A user who enters any skill should, within 30 minutes of using SIE, have at least one live, deployable income asset in the market — plus a personalised 30-day action plan to earn their first ₹10,000.

1.5 Technology Stack (100% Free Resources)

Component	Technology / Tool
LLM (Skill Decomposition + Content Generation)	Mistral-7B or LLaMA-3 (open-source; Colab / HF Spaces)
Web Scraping	Python + BeautifulSoup + Playwright
Database	PostgreSQL (free tier) / SQLite for local dev
Backend API	FastAPI (Python)
Frontend	React.js + Tailwind CSS
Landing Page Hosting	GitHub Pages / Vercel (free tier)
Portfolio Hosting	GitHub

Component	Technology / Tool
Gig Deployment	Fiverr Developer API + Upwork API
Tracking & Analytics	Custom UTM tracking + Plausible Analytics
Deployment	Railway.app / Render.com (free tier)

1.6 Why This Matters for Execution

Keep this snapshot in mind while working through the plan below: every phase exists to deliver one of these five modules, on this stack, toward that 30-minute / ₹10,000 goal.

2. Team Structure & Ownership

Role-to-name mapping is not finalised yet. The plan below uses Member 1 / Member 2 / Member 3 as placeholders for the three ownership tracks — assign actual names once decided, everything else stays the same.

2.1 Ownership Tracks

Track	Primary Ownership
Member 1	AI + Backend + Overall Integration
Member 2	Frontend + UI/UX
Member 3	Data + Scraping + Deployment + DevOps

2.2 Golden Principle

Nobody works completely independently.

Every member should understand, at least at a working level:

- The entire architecture
- The folder structure
- The APIs
- The database
- The deployment process
- Git workflow

Otherwise integration becomes a nightmare at the end.

3. Master Execution Roadmap

Think of the project like constructing a building — each layer depends on the one below it being stable.

1. Planning
2. GitHub Setup
3. Architecture
4. Frontend
5. Backend
6. Database
7. AI
8. Scrapers
9. Integration
10. Testing
11. Deployment
12. Documentation
13. Demo

4. Phase-by-Phase Development Plan (Phases 1–23)

Phase 1 — Project Planning (Week 1)

Owner: Entire Team

- Final Features
- Screens
- Workflow
- Tech Stack
- APIs
- Database
- Folder Structure
- Coding Standards
- Naming Standards
- Git Workflow

✓**Deliverable:** Everyone should be able to answer: "How does SIE work from Login until Dashboard?" No coding — only planning.

Phase 2 — Create GitHub Organization

Owner: Member 1

- Create the SIE repository
- Create sub-areas: Frontend, Backend, Documentation, Designs
- Create README.md, LICENSE, .gitignore, Contribution Guide, Project Wiki

✓**Deliverable:** Everyone clones the repo and creates branches: main, develop, feature/frontend, feature/backend, feature/ai, feature/scrapper, feature/dashboard. Never push directly to main.

Phase 3 — Decide Folder Structure

Owner: Entire Team

- frontend/
- backend/
- database/
- scrapers/
- ai/
- docs/
- deployment/
- assets/
- tests/
- .github/
- README.md

✓**Deliverable:** This structure is binding for the entire team.

Phase 4 — Design System

Owner: Member 2

- Logo
- Colors
- Buttons
- Cards
- Navbar
- Sidebar
- Dashboard
- Forms
- Fonts
- Icons
- Spacing
- Responsive layouts

✓**Deliverable:** Complete Figma file — everyone knows how every page looks. No confusion later.

Phase 5 — Database Design

Owner: Member 1 + Member 3

- ER Diagram
- Tables
- Relationships
- Primary Keys
- Foreign Keys
- Indexes
- Users, Skills, Projects, Market Data, Income Kits, Analytics, History, Feedback, Deployments

✓**Deliverable:** PostgreSQL schema, SQL scripts, migration files.

Phase 6 — Backend Setup

Owner: Member 1

- FastAPI
- Environment Variables
- Logging
- Authentication (JWT)
- Folder Structure
- Swagger / API Documentation
- Error Handling
- Validation
- Database Connection
- Health Check

✓**Deliverable:** Backend server running.

Phase 7 — Frontend Setup

Owner: Member 2

- React
- Tailwind
- Routing
- Layouts
- Theme
- Components
- Authentication Pages
- Dashboard Pages
- Loader
- Sidebar
- Navbar
- Footer
- Cards
- Charts

✓**Deliverable:** Dummy UI ready.

Phase 8 — Scraper Module

Owner: Member 3

- Fiverr Scraper
- Upwork Scraper
- LinkedIn Jobs Scraper
- GitHub Trending Scraper
- IndiaMART Scraper
- Cleaning
- Deduplication
- Store in Database
- Scheduler / Cron Jobs
- Logging
- Retry Logic

✓**Deliverable:** Market database updates automatically.

Phase 9 — AI Module

Owner: Member 1

- Skill Decomposition
- Prompt Templates
- LLM Integration
- Ranking
- Opportunity Generator

- Income Kit Generator
- Cold Email Generator
- Landing Page Generator
- Gig Generator
- GitHub README Generator

✓**Deliverable:** Everything returns JSON. Never HTML.

Phase 10 — APIs

Owner: Member 1

- Login
- Signup
- Skills
- Analyze Skill
- Market
- Dashboard
- Analytics
- Projects
- Deploy
- Feedback
- History

✓**Deliverable:** Document every endpoint.

Phase 11 — Frontend Integration

Owner: Member 2

- Connect React → Backend → Display

✓**Deliverable:** No business logic inside frontend — only API calls.

Phase 12 — Dashboard

Owner: Member 2

- Overview
- Skill Score
- Market Trends
- Top Opportunities
- Generated Assets
- Analytics
- Timeline
- Charts
- Dark Mode
- Export

✓**Deliverable:** Full dashboard experience.

Phase 13 — Analytics Module

Owner: Member 3

- Store Clicks, Views, Downloads, Generated Kits
- Most Selected Skills
- Trending Categories
- Growth
- Charts
- Statistics

✓**Deliverable:** Analytics pipeline feeding the dashboard.

Phase 14 — Authentication

Owner: Member 1

- Signup
- Login
- JWT
- Protected Routes
- Logout
- Forgot Password
- Email Verification (optional)

✓**Deliverable:** Secure auth flow end-to-end.

Phase 15 — Notifications

Owner: Member 2

- Toast Messages
- Loading
- Errors
- Warnings
- Success

✓**Deliverable:** Consistent feedback UI across the app.

Phase 16 — Testing

Owner: Entire Team

- Frontend
- Backend
- API
- Database
- Authentication
- Scraper
- LLM
- Deployment

- Responsive UI
- Edge Cases

✓**Deliverable:** Bugs fixed.

Phase 17 — Deployment Preparation

Owner: Member 3

- Dockerfile
- docker-compose
- Environment Variables
- Production Build
- Secrets
- CORS
- HTTPS
- Domain

✓**Deliverable:** Deployment-ready artifacts.

Phase 18 — Deploy Backend

Owner: Member 3 (Render / Railway)

- FastAPI
- Database
- Environment Variables
- Logs
- Health Check

✓**Deliverable:** Backend live in production.

Phase 19 — Deploy Frontend

Owner: Member 3 (Vercel)

- React
- Environment Variables
- Production Build
- Custom Domain

✓**Deliverable:** Frontend live in production.

Phase 20 — Connect Production

Owner: Entire Team

- Frontend → Production Backend → Production Database → Production Scrapers → Production AI → Production Analytics

✓**Deliverable:** Everything live and talking to each other.

Phase 21 — CI/CD

Owner: Member 3

- GitHub Actions
- Automatic Build
- Test
- Deploy after merge

✓**Deliverable:** Automated pipeline from merge to production.

Phase 22 — Documentation

Owner: Entire Team

- README
- Installation
- Folder Structure
- Architecture
- API Docs
- Screenshots
- Flow Diagram
- ER Diagram
- Sequence Diagram
- Deployment Guide
- User Manual
- Developer Guide

✓**Deliverable:** Complete, submission-ready documentation.

Phase 23 — Final Demo

Owner: Entire Team

- Presentation
- Architecture
- Flow
- Code
- Live Demo
- Deployment
- Results
- Future Scope
- Questions

✓**Deliverable:** Team is ready for evaluation.

5. Responsibility Matrix

✓= shared responsibility across the whole team ★= primary owner

Task	Member 1	Member 2	Member 3
Project Setup	✓	✓	✓
GitHub	✓	✓	✓
Folder Structure	✓	✓	✓
Backend	★		
FastAPI	★		
Authentication	★		
Database	★		★
AI Modules	★		
Prompt Engineering	★		
APIs	★		
Frontend		★	
React		★	
Tailwind		★	
UI/UX		★	
Dashboard		★	
Charts		★	
Responsive Design		★	
Scrapers			★
Data Cleaning			★
Scheduler			★
Analytics			★
Deployment			★
Docker			★
CI/CD			★
Testing	✓	✓	✓
Documentation	✓	✓	✓
Demo	✓	✓	✓

6. SIE Master Execution Checklist

Step 0 — Common Setup (All Members)

Goal: everyone starts from the same environment.

- ☒ Finalize project requirements
- ☒ Finalize tech stack
- ☒ Create GitHub repository
- ☒ Add all members as collaborators
- ☒ Create README
- ☒ Create .gitignore
- ☒ Create folder structure
- ☒ Create Git branches
- ☐ Decide coding standards
- ☐ Decide API naming
- ☐ Decide database naming
- ☒ Decide folder naming
- ☐ Decide commit message format
- ☒ Setup local environment
- ☐ Install required software
- ☐ Verify project runs locally

Member 1 — AI + Backend Lead

Phase 1 — Backend Foundation

- ☒ Setup FastAPI project
- ☒ Create backend folder structure
- ☒ Configure environment variables
- ☐ Connect PostgreSQL
- ☐ Configure ORM (SQLAlchemy)
- ☐ Create authentication system
- ☐ JWT Login
- ☐ Signup
- ☐ Password hashing
- ☐ Role management
- ☒ Logging
- ☐ Exception handling
- ☒ Swagger documentation

✓ **Deliverable: Backend server running.**

Phase 2 — Database APIs

- ☐ User APIs

- ☐ Skill APIs
- ☐ Market APIs
- ☐ Analytics APIs
- ☐ History APIs
- ☐ Feedback APIs
- ☐ Dashboard APIs

✓**Deliverable: All CRUD APIs.**

Phase 3 — AI Engine

- ☐ Prompt Engineering
- ☐ Skill decomposition
- ☐ Opportunity ranking
- ☐ Income Kit generation
- ☐ Gig generator
- ☐ README generator
- ☐ Landing page generator
- ☐ Outreach generator

✓**Deliverable: AI returns JSON.**

Phase 4 — Integration

- ☐ Connect database
- ☐ Connect scraper data
- ☐ Connect frontend
- ☐ API testing
- ☐ Performance optimization

✓**Deliverable: Backend complete.**

Member 1 — Dependency Checklist (before moving forward)

Before writing any code, verify:

- ☐ Database schema finalized by Member 3
- ☐ UI fields finalized by Member 2
- ☐ API contract shared with frontend
- ☐ JSON response format frozen
- ☐ Table names not changing
- ☐ Environment variables documented
- ☐ Swagger updated after every API change

Member 2 — Frontend Lead

Phase 1 — UI Design

- ☐ Figma design
- ☐ Theme
- ☐ Color palette
- ☐ Typography
- ☐ Icons
- ☐ Responsive layout
- ☐ Component library

✓**Deliverable:** Complete UI design.

Phase 2 — React Setup

- ☐ React project
- ☐ Tailwind setup
- ☐ Routing
- ☐ Global Layout
- ☐ Navbar
- ☐ Sidebar
- ☐ Footer

✓**Deliverable:** Frontend skeleton.

Phase 3 — Pages

- ☐ Login
- ☐ Signup
- ☐ Dashboard
- ☐ Skill Analysis
- ☐ Market Insights
- ☐ Opportunity Page
- ☐ Income Kit
- ☐ Analytics
- ☐ Settings

✓**Deliverable:** UI complete.

Phase 4 — API Integration

- ☐ Login API
- ☐ Dashboard API
- ☐ Skill API
- ☐ Analytics API
- ☐ Loading states
- ☐ Error states
- ☐ Success messages

✓**Deliverable:** Frontend connected.

Member 2 — Dependency Checklist

Before coding, verify:

- ☐ API documentation received from Member 1
- ☐ JSON response format finalized
- ☐ Endpoint URLs frozen
- ☐ Authentication flow finalized
- ☐ Field names confirmed
- ☐ Error codes documented
- ☐ Dashboard data format confirmed

Never hardcode values that come from the backend.

Member 3 — Data + Scraping + DevOps

Phase 1 — Database

- ☐ Design ER diagram
- ☐ Create schema
- ☐ Relationships
- ☐ Indexes
- ☐ Seed data
- ☐ Migration scripts

✓**Deliverable:** Database ready.

Phase 2 — Scraping Engine

- ☐ Fiverr scraper
- ☐ Upwork scraper
- ☐ LinkedIn scraper
- ☐ GitHub Trending scraper
- ☐ Data cleaning
- ☐ Duplicate removal
- ☐ Store in database

✓**Deliverable:** Market data pipeline.

Phase 3 — Scheduler

- ☐ Cron jobs
- ☐ Automatic updates
- ☐ Logs
- ☐ Retry mechanism

✓**Deliverable:** Automated scraper.

Phase 4 — Deployment

- ☐ Docker
- ☐ Docker Compose
- ☐ Backend deployment
- ☐ Frontend deployment
- ☐ Database deployment
- ☐ Domain
- ☐ HTTPS
- ☐ Environment variables
- ☐ CI/CD
- ☐ Monitoring

✓**Deliverable:** Live application.

Member 3 — Dependency Checklist

Before coding, verify:

- ☐ Required tables approved by Member 1
- ☐ Required UI fields approved by Member 2
- ☐ API endpoints available
- ☐ Scraper output format documented
- ☐ Deployment environment agreed upon
- ☐ Secrets stored securely
- ☐ Docker versions compatible across team

7. Final Integration Checklist (All Members)

Before merging into main:

- ☐ No merge conflicts
- ☐ Latest develop branch pulled
- ☐ Code builds successfully
- ☐ All APIs working
- ☐ Database migrations applied
- ☐ Frontend connected
- ☐ All responses validated
- ☐ Scrapers storing data correctly
- ☐ Authentication working
- ☐ Dashboard loading live data
- ☐ Environment variables configured
- ☐ README updated
- ☐ Tests passed
- ☐ Code reviewed by at least one teammate

8. Daily Team Checklist (15-Minute Stand-up)

Each member answers, every day:

- ☐ What did I complete yesterday?
- ☐ What am I working on today?
- ☐ What is blocking me?
- ☐ Did I push my code?
- ☐ Did I raise a Pull Request?
- ☐ Did I update the documentation?
- ☐ Did I change any API?
- ☐ Did I change any database table?
- ☐ Did I inform the other members about those changes?
- ☐ Is my code still compatible with everyone else's work?

9. Project Completion Checklist

Before submitting or deploying:

- ☐ GitHub repository organized
- ☐ Clean commit history
- ☐ Backend fully functional
- ☐ Frontend fully responsive
- ☐ Database optimized
- ☐ AI pipeline working
- ☐ Scraper pipeline working
- ☐ Dashboard displaying live data
- ☐ Deployment successful
- ☐ Documentation complete
- ☐ Architecture diagrams included
- ☐ API documentation complete
- ☐ User manual prepared
- ☐ Demo script rehearsed
- ☐ Final presentation ready

This checklist gives every member a clear scope while making dependencies explicit, reducing the chance of incompatible code or integration issues. It mirrors a simplified version of how real software teams manage large projects in industry.

10. Team Collaboration Timeline & Synchronization Gates

Real software teams don't work in complete isolation from start to finish. Instead, there are “Synchronization Gates” (integration checkpoints) where everyone pauses, reviews dependencies, and then continues. Below is the complete execution flow for SIE.

10.1 Flow Overview

Stage 1 — All Together

- Planning

Stage 2 — Can Work Independently

- Backend Setup
- Frontend Setup
- Database Setup

— FIRST SYNC —

- Authentication
- Database Schema
- API Contract

Stage 3 — Individual Work

- Backend APIs
- Frontend Pages
- Scrapers
- AI Module

— SECOND SYNC —

- API Review
- JSON Format
- Database Review

Stage 4 — Individual Work

- API Integration
- Dashboard
- Analytics

— THIRD SYNC —

- Integration Testing

Stage 5 — Final Individual Work

- Bug Fixes
- Deployment
- Documentation

— FINAL TEAM SYNC —

- Live Deployment
- Testing

- Presentation

10.2 Mandatory Team Meetings (Synchronization Gates)

Sync Gate	When?	Duration	Everyone Must Bring
Gate 1 – Planning Freeze	Before coding starts	2–3 hrs	Requirements, architecture, Git setup, folder structure
Gate 2 – Design Freeze	After UI & DB design	1–2 hrs	Figma, ER diagram, finalized database schema
Gate 3 – API Freeze	Before frontend integration	2 hrs	Swagger docs, request/response JSON, endpoint list
Gate 4 – Integration Check	After AI, backend, scraper complete	2 hrs	Working APIs, scraper output, AI JSON responses
Gate 5 – System Test	Before deployment	Half day	Full end-to-end testing, bug list
Gate 6 – Release Freeze	Before final demo	1–2 hrs	Deployment verification, documentation, presentation

11. Master Reference — Full 30-Step Phase Table

This is the single most complete reference table: every step, who touches it, whether it can be done independently, and whether a team sync is mandatory before moving on.

#	Task	M1	M2	M3	Independent?	Sync?	What Must Be Checked
1	Requirement Analysis	✓	✓	✓	✗No	✓Yes	Everyone agrees on project scope, modules, features
2	Tech Stack Finalization	✓	✓	✓	✗No	✓Yes	Same versions, libraries, deployment tools
3	GitHub Setup	✓	✓	✓	✗No	✓Yes	Repository, branches, folder structure
4	Coding Standards	✓	✓	✓	✗No	✓Yes	Naming conventions, API format, commits
5	UI Wireframes (Figma)	Observe	*Build	Observe	✓Mostly	✓Yes	Backend & DB review required so UI contains required fields
6	Database Design	Review	Review	*Build	✗No	✓Yes	Tables, columns, IDs, relationships finalized
7	API Contract Design	*Build	Review	Review	✗No	✓Yes	Endpoint names, request/response JSON frozen
8	Backend Project Setup	*			✓Yes	✗No	Basic FastAPI setup only
9	Frontend Project Setup		*		✓Yes	✗No	Basic React setup only
10	Database Setup			*	✓Yes	✗No	PostgreSQL, ORM, migrations
11	Authentication Module	*	UI Screens	DB Support	△Partial	✓Yes	Login response format, JWT flow
12	Basic Dashboard UI	API Mock	*	Mock Data	✓Yes	✗No	Use dummy data temporarily
13	Scraper Development			*	✓Yes	✗No	Store data in agreed database schema
14	AI Module Development	*		Uses DB	✓Yes	✗No	AI returns agreed JSON structure
15	Landing Page Templates	AI Content	*UI		✓Yes	✗No	Static templates only
16	Income Kit Templates	*	*		△Partial	✓Yes	JSON format matches frontend
17	API Development Complete	*		DB Ready	△Partial	✓Yes	Swagger reviewed by all members
18	Frontend API Integration	API Ready	*		✗No	✓Yes	Every endpoint tested

#	Task	M1	M2	M3	Independent?	Sync?	What Must Be Checked
19	AI + Backend Integration	★	UI Ready	DB Ready	✗No	✓Yes	AI connected to database
20	Scraper + Backend Integration	★		★	✗No	✓Yes	Market data reaching backend
21	Dashboard Live Data	API	★	Database	✗No	✓Yes	Charts display real data
22	Analytics Module	API	Charts	★	△Partial	✓Yes	Analytics format confirmed
23	Complete System Testing	✓	✓	✓	✗No	✓Yes	Entire application tested together
24	Bug Fix Sprint	✓	✓	✓	△Partial	✓Yes	Merge conflicts, API changes
25	Docker Setup			★	✓Yes	✗No	Backend/frontend configuration documented
26	Backend Deployment	★		★	△Partial	✓Yes	Production API URL shared
27	Frontend Deployment	API Ready	★		✗No	✓Yes	Backend URL updated
28	Database Deployment			★	△Partial	✓Yes	Production DB connected
29	End-to-End Testing	✓	✓	✓	✗No	✓Final	Live application verification
30	Documentation & Demo	✓	✓	✓	✗No	✓Final	README, diagrams, presentation

12. Golden Rules for the Team

Whenever one of these situations comes up, the corresponding action is mandatory — not optional.

Situation	Action
Database table changes	Inform Member 1 and Member 2 before changing
API request/response changes	Inform all members immediately
New UI field added	Confirm backend API and database support it
AI JSON format changes	Frontend must verify parsing before merge
Scraper output changes	Backend must validate before storing
Environment variable changes	Update .env.example and notify everyone
Git merge before main	At least one teammate reviews the Pull Request
Before starting a new feature	Check if it depends on another member's unfinished work
Before ending the day	Push code, update progress, communicate blockers

13. Scope Recommendation — Keep It Achievable

SIE's full documentation describes a very ambitious product, including automated deployment to freelance platforms and a self-learning feedback loop. For a final-year project, it's strongly recommended to split delivery into two milestones so the core is guaranteed to work end-to-end before layering on the advanced pieces.

Phase 1 — Core MVP

User login → Skill analysis → Market intelligence → Opportunity ranking → Income Kit generation → Dashboard.

This demonstrates the complete end-to-end pipeline and is enough on its own for evaluation.

Phase 2 — Advanced Features

Automated deployment to Fiverr/Upwork, A/B testing, click analytics, adaptive recommendations, scheduled scraping, and continuous feedback.

This keeps the project achievable while leaving room to implement the advanced capabilities if time permits, without risking an incomplete core system.

How to Use This as an SOP

Treat this whole handbook as the project's Standard Operating Procedure. Every member has a clear checklist, knows when to stop and sync with the team, what to deliver at each stage, and what dependencies to check before moving ahead.